HALLAZGO TCK - 001 Pendiente Validación

Plantilla: Plantilla Profesional de Hallazgo (Ticket)
Ref ID: TCK - 001

Curso ciberseguridad 2026 v1.0.0

TÍTULO DE LA VULNERABILIDAD

Timing Attack

SEVERIDAD GLOBAL

Media

ACTIVO / ENDPOINT AFECTADO

https://chl-964a194a-d54c-4485-9993-00310404311d-asistencia.softwareseguro.com.ar/

CLASIFICACIÓN CWE

CWE-208

MÉTRICA CVSS V3.1

5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)

RESUMEN EJECUTIVO (PARA DIRECTIVOS)

Durante la auditoría de seguridad sobre la plataforma de control de asistencia, se identificó un fallo que permite a un atacante externo y sin credenciales averiguar con total exactitud la cantidad y los códigos de los asistentes registrados en el evento.

Aunque el sistema muestra siempre el mismo mensaje genérico frente a intentos fallidos ("No se puede acceder a esta información"), el servidor web demora notablemente más tiempo en responder cuando el código ingresado existe en la base de datos comparado con uno inexistente. Esta fuga de información vulnera la privacidad del padrón de asistentes y permite a terceros reconstruir la asistencia real del evento sin autorización.

DETALLES TÉCNICOS Y CAUSA RAÍZ

El formulario web solicita dos campos: codigo (restringido a 3 dígitos numéricos [0-9]{3}, del 000 al 999) y clave.

El problema radica en que el backend valida secuencialmente los datos con una salida temprana (early exit):
1. Si el codigo ingresado no existe en la base de datos, el servidor rechaza la petición casi de inmediato (tardando aproximadamente 0.18s a 0.20s).
2. Si el codigo sí existe, el backend procede a verificar la clave ingresada, aplicando una función de cómputo costosa (como un hash de contraseñas tipo Bcrypt o un retardo interno). Esto provoca que la respuesta tarde entre 1.10s y 1.25s.

Esa diferencia sistemática de más de 800 milisegundos crea un canal lateral (side-channel timing attack) que permite a un atacante diferenciar con absoluta certeza qué códigos están en uso y cuáles no.

PRUEBA DE CONCEPTO (POC) PASO A PASO

Precondiciones:
Conexión de red hacia el endpoint del laboratorio. Intérprete de Python con la librería requests instalada.

Pasos de Reproducción:
1. Inspeccionar el formulario de la aplicación web y constatar la restricción de 3 dígitos (000 a 999).
2. Enviar peticiones POST manteniendo un valor de clave fijo cualquiera (ej. clave: "test"), variando el campo codigo y registrando el tiempo de respuesta:
   - Código 158: 0.24s (Inexistente)
   - Código 159: 0.18s (Inexistente)
   - Código 160: 1.27s (Lento -> Código Asignado)
   - Código 161: 1.16s (Lento -> Código Asignado)
   - Código 162: 0.20s (Inexistente)
   - Código 163: 1.26s (Lento -> Código Asignado)

3. Automatización mediante script en Python:
   Se desarrolló un script multihilo (ThreadPoolExecutor) que itera los 1000 códigos posibles con clave fija y mide res.elapsed.total_seconds(). Aquellos códigos cuyo tiempo supera el umbral de 0.9 segundos se contabilizan como asistentes activos. Al finalizar el escaneo, la cantidad total de códigos lentos representa el número exacto de asistentes registrados en el sistema.

RECOMENDACIONES DE REMEDIACIÓN

Solución Definitiva (Tiempo de Respuesta Constante):
Modificar el backend para que el proceso de autenticación tome siempre un tiempo uniforme independientemente de si el código existe o no. Si el código ingresado no existe en la base de datos, el sistema no debe retornar de inmediato; debe ejecutar una operación de hash ficticia (dummy hash) con el mismo costo computacional antes de devolver el mensaje de error.

Mitigación Complementaria:
1. Rate Limiting: Implementar un límite estricto de peticiones por dirección IP (ej. máximo 5 intentos por minuto) para neutralizar escaneos automatizados.
2. Captcha / Bloqueo: Incorporar un desafío visual tras detectar múltiples intentos erróneos consecutivos.